

Lab1:DataLab 何梓昕 25303050286

截图：

```
roundEvenPow2: 11/24 operators
midpointTowardFirst: 24/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 35/52 operators
floatScaleThreeHalves: 44/60 operators
floatRoundEven: 46/65 operators
float_i2f: 33/40 operators
bitCount: 34/40 operators
bitReverse: 32/34 operators
All 19 functions passed operator checks.
```

```
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
hzixin@Bu:/mnt/c/Users/honor/Desktop/DataLab_hzixin$
```

实验思路：

P1 signMask: 利用 $1 \ll 31$ 构造最高位为 1 的掩码 0x80000000。

P2 bitXor: 利用德摩根定律，将异或等价转化为 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$ 。

P3 negativePart: 通过 $x \gg 31$ 获取全 1 或全 0 的符号掩码。利用补码求相反数公式 $\sim x + 1$ ，结合掩码 & 实现“负数返回相反数，正数返回 0”。

P4 copyByteWithin: 先提取 src 字节，再将 dst 字节清零，最后通过 | 拼装。

P5 logicalShift: 通过构造掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 将算术右移补进来的高 n 位 1 擦除。

P6 swapNibblePairs: 利用 0x0F 逐级左移拼凑出 0x0F0F0F0F 掩码，分别提取高低 4 位并交换移位，最后用 | 拼装。

P7 secondLowestZeroBit: 利用公式 $\sim x \& (x + 1)$ 提取最低位的 0，将其置为 1 后，再次套用公式提取新的最低位 0。

P8 oddParity: 利用异或的“无进位加法”和结合律，将 32 位分段（每 4 位）提取异或，最后将结果逐步折叠 ($\gg 2$ 和 $\gg 1$) 至最低位，! 取反得到奇偶校验位。

P9 rotateRightBits: 循环右移 n 位等价于 逻辑右移 n | 左移 $(32-n)$ 。利用 $(\sim n + 1) \& 31$ 替换 $32-n$ 并规避 $n=0$ 时的移位未定义行为。

P10 roundEvenPow2: 利用 $x + \text{bias} + \text{lsb}$ 的进位机制。 $\text{bias} = 2^{(n-1)} - 1$ 确保正中间默认舍去， $\text{lsb} = (x \gg n) \& 1$ 在商为奇数时补 1 进位，实现向偶数舍入。

P11 midpointTowardFirst: 用 $\text{half_x} + \text{half_y} + (x \& y \& 1)$ 安全计算均值，防止 $x+y$ 溢出。通过分类讨论（同号/异号）安全计算 $x-y$ 的符号位，用 $(\text{mask} \& A) | (\sim \text{mask} \& B)$ 选出正确的修正值。

P12 isBetweenEitherOrder: 安全计算 $x-a$ 和 $x-b$ 的符号位。若符号位不同（异或得全 1），则 x 在区间内。最后用 $!(x^a) | !(x^b)$ 补丁处理端点情况。

P14: 核心思路是采用“分步相加 + 分类讨论”：

首先安全计算 $x + y$ ，并利用符号位异或逻辑判断是否溢出。若溢出，需推导出当前结果与真实数学和的“溢出量 k ”（正溢出为 $\text{INT_MAX} + 1 + k$ ，负溢出为 $\text{INT_MIN} - 1 - k$ ）。

随后将 k 与 z 进行安全比较。例如，在正溢出情况下，如果 $z \geq -k$ ，说明数学和依然大于 INT_MAX ，返回 1；否则返回 -1。非溢出情况则继续与 z 进行安全加法并再次判断溢出。

P15: 补上隐藏位 (0x800000) 后乘以 3。根据乘法结果是否达到进位阈值（如 0x2000000），决定阶码是否加 1。若产生进位，尾数需除以 4 并进行向偶数舍入；若无进位，尾数除以 2 并进行向偶数舍入。随后需重新分离出规格化的尾数，并处理尾数进位导致的二次阶码溢出（变为无穷大）。

P16: 通过真实指数 E 进行数轴分类讨论：

$E < -1$ （绝对值 < 0.5 ），直接返回带符号的 0；

$E == -1$ （绝对值在 $[0.5, 1.0)$ ），若尾数为 0（刚好 0.5）向偶数舍入返回 0，否则进位到 1.0；

$E \geq 23$ ，本身即为整数，原样返回；

常规区间 $0 \leq E < 23$ ，提取尾数并截断，根据被砍掉的位判断是否进位：大于一半进位，刚好一半时看向偶数舍入规则（看保留位最后一位是否为奇数）。

特别处理：在 $E == 0$ （绝对值在 $[1.0, 2.0)$ ）区间，整数部分隐藏的 1 是奇数，但 frac 右移后会被清零，导致丢失奇偶性判断。因此该区间被单独提取，若 $\text{frac} \geq 0x400000$ （即 ≥ 1.5 ）直接进位到 2.0，否则保持 1.0。

P17 float_i2f: 提取符号，安全取绝对值，while 循环找最高有效位算阶码 $e+127$ 。尾数处理时，若 $e > 23$ 需右移砍位，利用 $\text{out} > \text{half}$ 或 $== \text{half}$ 配合 $\text{frac} \& 1$ 实现向偶数舍入，最后处理尾数进位溢出（ $\text{frac} == (1 \ll 23)$ ）并拼装。

P18 bitCount: 。构造 0x11111111 掩码将每 4 位作为一个整体，利用 $(x \gg 1) \& \text{mask1}$ 等操作把不同位的 1 累加到 4 位空间中。随后逐步将 4 位整体合并为 8 位、16 位、32 位整体，得到总数。

P19 bitReverse: 依次进行相邻 1 位、2 位、4 位、8 位、16 位的交换。使用掩码（如 0x55555555、0x33333333）配合 $\gg$ 和 $\ll$ 实现并行翻转。

对实验的感受和对助教的建议：

个人认为这 19 题中 14、15、16 题最难，甚至对比前后的题目有一种难度一下子跃升的感觉。建议以后可以将难度最高的题目统一放在整个 lab 的最后。（甚至有点太难了，是我的问题吗……）